

Abstract

Industrial machinery accounts for a significant share of operational costs in manufacturing, and unplanned downtime due to mechanical failure can result in catastrophic financial and safety consequences. This paper presents the design and implementation of a low-cost, fully embedded Predictive Maintenance system that uses **TinyML** to detect mechanical anomalies in industrial equipments entirely on the edge — without cloud connectivity or a host PC.

An **ESP32-S3 (N16R8) microcontroller** collects three-axis vibration data from an **MPU6500 accelerometer** at 100 Hz, feeds it through a Spectral Analysis DSP pipeline, and runs two parallel AI models: a **Neural Network Classifier** (distinguishing "Normal" operation from "Loose Bolt") and a **K-Means Anomaly Detector** (flagging unknown fault patterns never seen during training). The models, trained on the Edge Impulse platform and compiled using the EON™ Compiler, execute in under **1 ms** while consuming only **1.5 KB of RAM and 16.4 KB of Flash**.

The quantized (int8) classifier achieves **94.74% test accuracy**, and the float32 baseline reaches 96.05%. Inference results are displayed in real time on an **SSD1306 OLED display**. The system demonstrates that production-grade, real-time AI fault detection is achievable within the power and resource budget of a microcontroller costing under ₹2,500.

Source Code & Project Files:

github.com/Rajneesh315/Predictive-Maintenance-System-on-ESP32-S3

Table Of Content

Abstract	1
1. Introduction.....	1
1.1 Background of Predictive Maintenance	1
1.2 Role of Edge AI and TinyML in Equipment Monitoring	1
1.3 Project Objectives	2
2. Hardware Architecture.....	3
2.1 Microcontroller – ESP32-S3.....	3
2.2 Sensors and Display.....	3
2.3 I2C Bus Design and Clock Speed Configuration.....	3
3. Data Collection & Digital Signal Processing (DSP)	5
3.1 Test Rig and Experimental Setup.....	5
3.2 Data Collection via Edge Impulse	5
3.3 Raw Time-Series Waveforms.....	6
3.4 Spectral Feature Extraction (DSP Block).....	7
4. Machine Learning Architecture	8
4.1 Brain 1 – Neural Network Classifier	8
4.2 Brain 2 – K-Means Anomaly Detector	10
5. Firmware Integration & Deployment	11
5.1 EON™ Compiler and Model Export	11
5.2 Firmware Logic	11
6. Results & Performance.....	13
6.1 Classification Accuracy	13
6.2 On-Device Performance	15
6.3 Physical Validation.....	15
7. Classification Accuracy	16
8. Future Scope.....	17
9. References.....	18

1. Introduction

1.1 Background of Predictive Maintenance

The rapid growth of industrial automation has significantly increased its reliance on the continuous, uninterrupted operation of machinery. Conventional maintenance strategies often rely on scheduled periodic check-ups or reactive repairs after a breakdown has occurred, both of which are inefficient and costly. Predictive maintenance focuses on monitoring the real-time physical health of equipment to predict and prevent mechanical failures before they happen, thereby minimizing downtime and optimizing resource utilization.

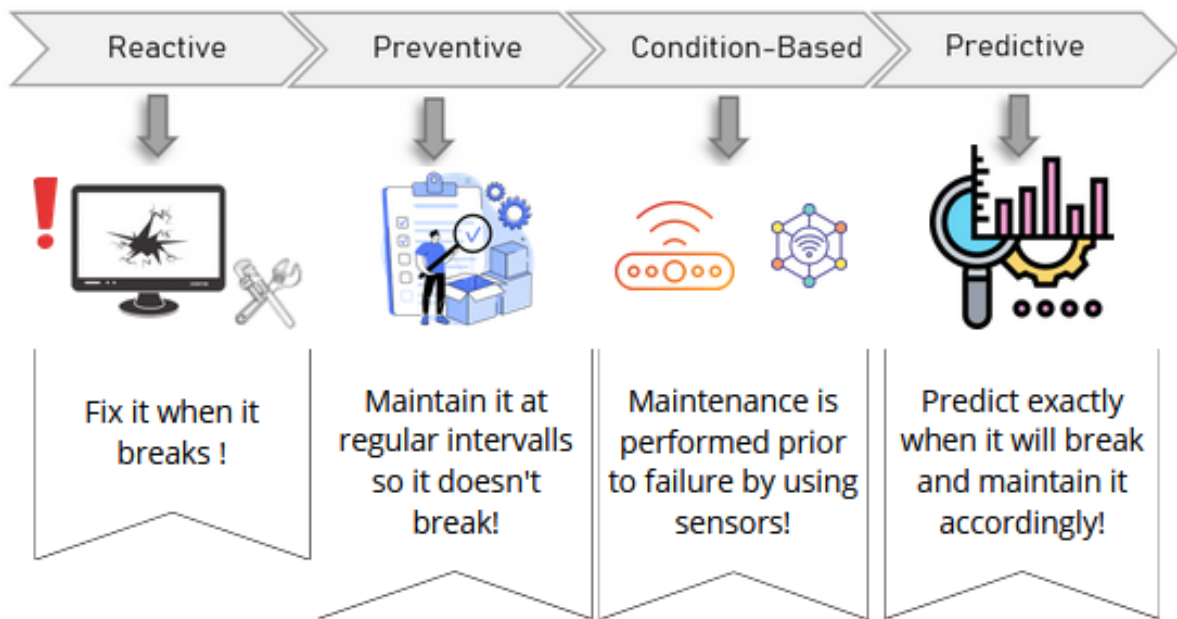


Figure 1 Evolution of Maintenance Systems

1.2 Role of Edge AI and TinyML in Equipment Monitoring

Conventional Predictive Maintenance solutions rely on *cloud-connected gateways*: sensors stream raw data to a remote server where heavy ML models run. While powerful, this architecture has serious limitations in industrial settings: network latency delays alarm response, cloud connectivity is unavailable in many factory floors, raw data transmission consumes bandwidth and raises privacy concerns, and recurring cloud subscription costs are prohibitive for SMEs.

TinyML — the practice of deploying optimised machine learning models directly on resource-constrained microcontrollers — eliminates all these drawbacks. Key advantages include:

- Zero-latency inference: decisions are made in microseconds, locally.
- Offline operation: no Wi-Fi or cellular connectivity required.
- Privacy by design: raw sensor data never leaves the device.
- Ultra-low power: the ESP32-S3 can run for hours on a small battery.
- Low cost: a complete node costs well under ₹2,500 in components.

1.3 Project Objectives

The objective of this project is to design, build, and validate a fully self-contained embedded PdM node (prototype) that:

- Continuously samples 3-axis vibration data from a rotating fan assembly.
- Runs a DSP spectral feature pipeline and dual AI inference loop on the microcontroller.
- Classifies the machine state as "Normal" or "Loose Bolt" using a Neural Network.
- Independently flags unknown anomalies using a K-Means clustering model.
- Displays the health status on a local OLED screen with zero cloud dependency.

2. Hardware Architecture

2.1 Microcontroller – ESP32-S3

The **ESP32-S3** (Xtensa LX7 dual-core, 240 MHz) was selected as the processing hub for several reasons that directly support TinyML deployment:

- 240 MHz dual-core CPU — fast enough to run DSP + neural network inference.
- 512 KB SRAM + 16 MB PSRAM (N16R8 variant) — sufficient to hold the inference buffer and model weights.
- Built-in USB-OTG — eliminates the need for an external USB-to-UART converter.
- Any GPIO pin can be used for I2C communication.
- Arduino IDE compatible — seamless integration with the Edge Impulse Arduino Library.

2.2 Sensors and Display

The **MPU6500** is a 6-DoF MEMS IMU (Inertial Measurement Unit) combining a 3-axis accelerometer and 3-axis gyroscope. For this project, only the accelerometer axes (X, Y, Z) are used, configured to $\pm 4g$ range — appropriate for the low-amplitude, high-frequency vibrations produced by a small fan. The accelerometer's digital I2C output makes it directly compatible with the ESP32-S3 without any external ADC.

The SSD1306 0.96" OLED (128×64 pixels, monochrome) provides local, real-time display of inference results. The low current draw (<10 mA at full brightness) makes it suitable for battery-powered deployment

2.3 I2C Bus Design and Clock Speed Configuration

A critical design decision was the use of two separate I2C buses for the two peripherals, rather than sharing a single bus. Both the MPU6500 and the SSD1306 default to I2C address 0x68 and 0x3C respectively. While their addresses do not conflict in this instance, sharing a bus with two time-sensitive devices — one for display refresh, one for sensor sampling — risks electrical interference and I2C arbitration delays that would corrupt the 100 Hz sampling rate. OLED is connected to the primary I2C bus (Wire) and set to 400 kHz (Fast Mode) for screen refresh. The MPU6500 uses the secondary bus (Wire1) at the conservative **100 kHz (Standard Mode)** to get stable data acquisition across longer wire.

Connection and circuit:

```
// OLED on Wire (GPIO 42/2) - 400 kHz for fast display
Wire.begin(OLED_SDA, OLED_SCL);
Wire.setClock(400000);
// MPU6500 on Wire1 (GPIO 11/12) - 100 kHz for stable data
Wire1.begin(MPU_SDA, MPU_SCL);
Wire1.setClock(100000);
```

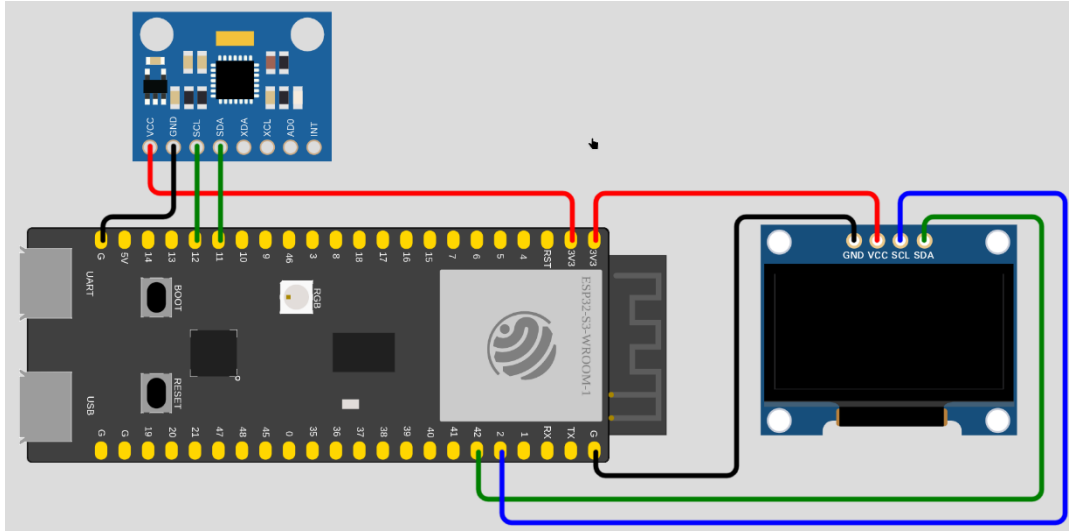


Figure 3 Circuit / Wiring Diagram – ESP32-S3, MPU6500, and OLED

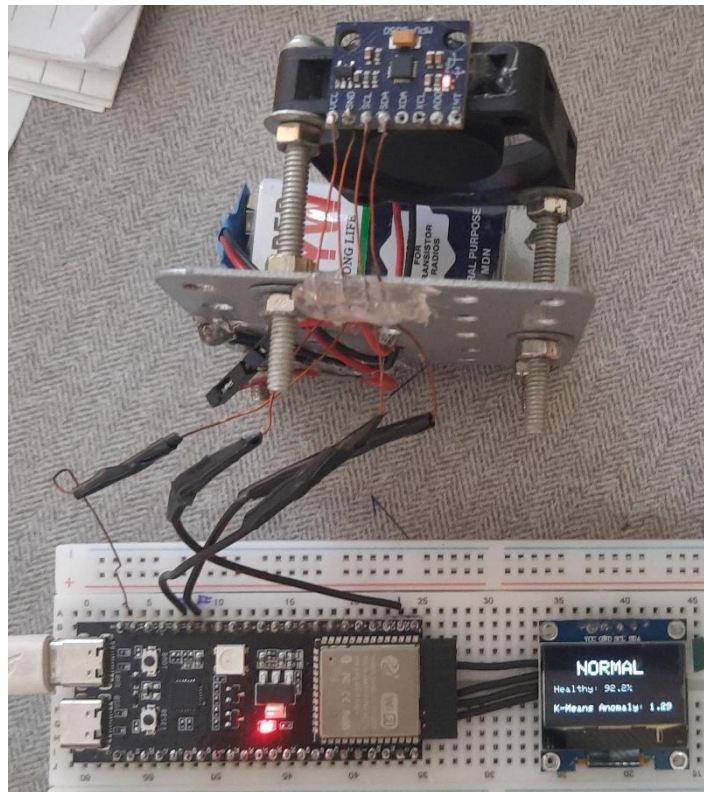


Figure 2 Physical test rig – fan under normal balanced operation

3. Data Collection & Digital Signal Processing (DSP)

3.1 Test Rig and Experimental Setup

The test apparatus consists of a small brushless DC fan mounted on a breadboard. Two machine states were recorded:

- "Normal" – All four mounting screws tightened; balanced, symmetrical vibration.
- "Bad" (Loose Bolt) – One mounting screw deliberately left loose; introduces an asymmetric vibration signature due to micro-impacts of the housing against the surface at every revolution.

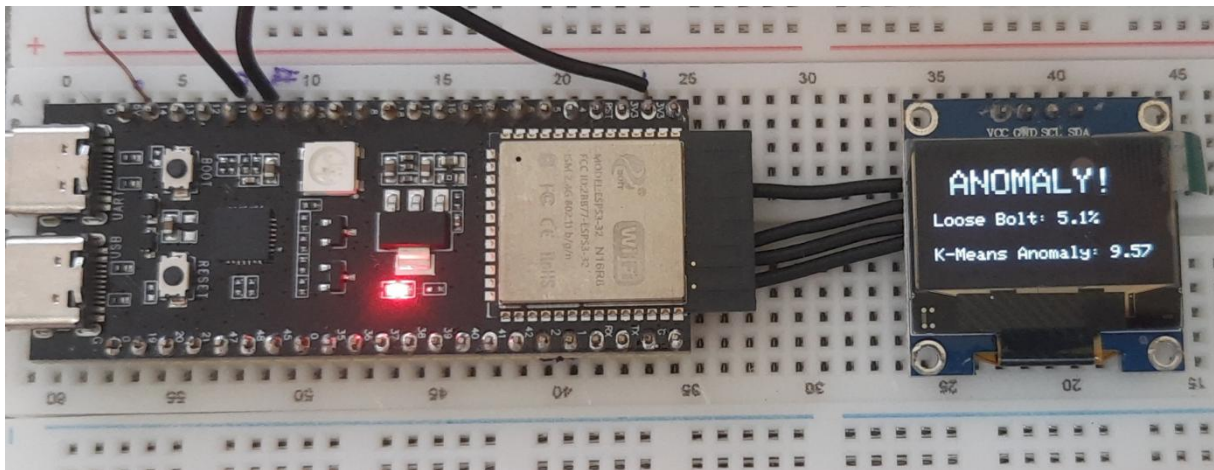


Figure 4 Physical test rig – fan with deliberately loosened bolt

3.2 Data Collection via Edge Impulse

Sensor data was streamed to the **Edge Impulse** cloud platform using the **Edge Impulse Data Forwarder** utility running over USB Serial. Each sample captures **10 seconds (10,000 ms)** of simultaneous accelX, accelY, and accelZ readings at a fixed rate of **100 Hz**, producing 1,000 samples per axis per recording. A total of **6 minutes and 40 seconds of data** was collected across both classes, split 80/20 between training (32 samples) and test (8 samples) sets.

Collect data  

Device 

esp32s3 n16r8 board 

Label

Sample length (ms.)

Sensor 

Frequency 



Figure 5 Edge Impulse Data Collection settings (100 Hz, 10 s window)

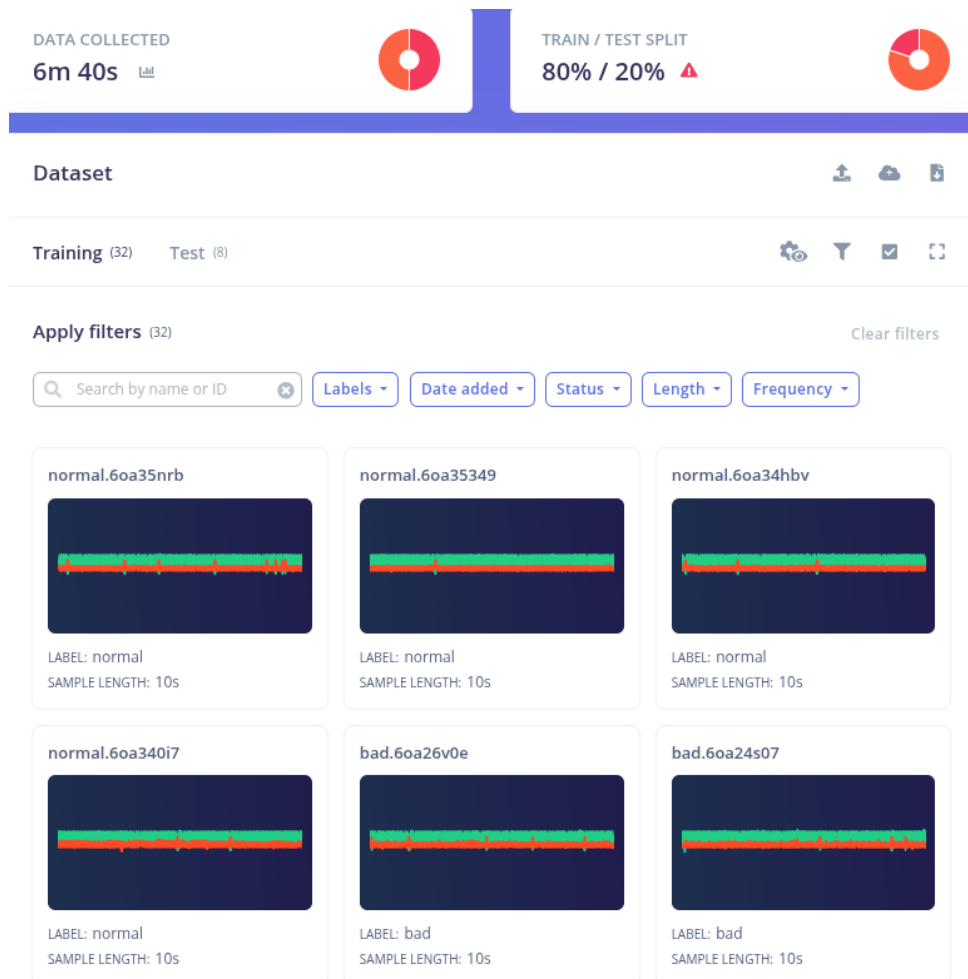


Figure 6 Acquired dataset – 40 samples, 80/20 train/test split

3.3 Raw Time-Series Waveforms

Inspection of the raw accelerometer data immediately reveals the difference between the two classes. The "Normal" signal is periodic and low amplitude. The "Bad" signal shows higher amplitude spikes at irregular intervals, reflecting the mechanical shock of the loose component.

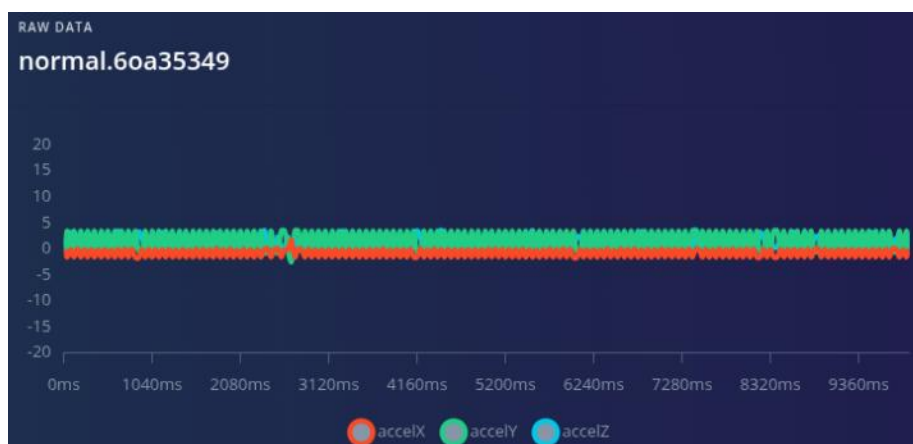


Figure 7 Raw accelerometer data – "Normal" (Healthy) condition

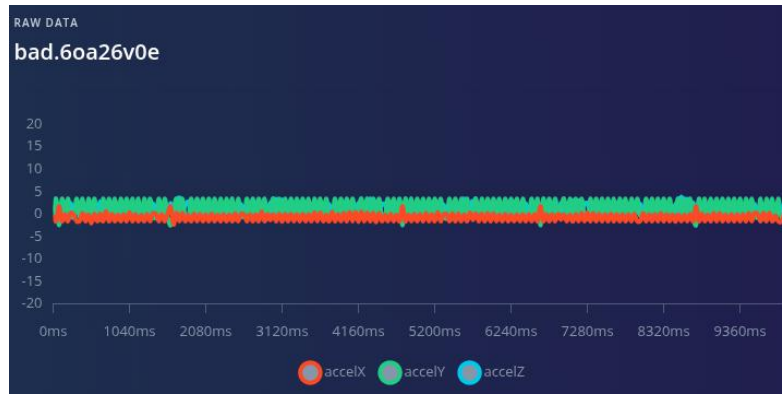


Figure 8 Raw accelerometer data – "Bad" (Loose Bolt) condition

3.4 Spectral Feature Extraction (DSP Block)

Raw time-series data is not fed directly to the neural network. Instead, a **Spectral Analysis DSP block** transforms it into a compact, frequency-domain feature vector. The pipeline operates on a **1,000 ms sliding window** with a **500 ms stride**, and performs the following signal processing steps:

- **Fast Fourier Transform (FFT):** Decomposes each axis's time-domain signal into its constituent frequency components, exposing the harmonic structure of the vibration.
- **RMS Energy:** Computes the Root Mean Square energy of each axis (accelX RMS, accelY RMS, accelZ RMS), capturing the overall vibration intensity — a key indicator of mechanical imbalance.
- **High-Pass Filter:** A high-pass filter strips out the DC component caused by Earth's gravitational acceleration (9.8 m/s^2 offset). This ensures the neural network learns only from dynamic vibration content, not static orientation.

The resulting feature vector of 108 floating-point values provides a rich, discriminative representation of the machine state that is well-suited for the neural network classifier.

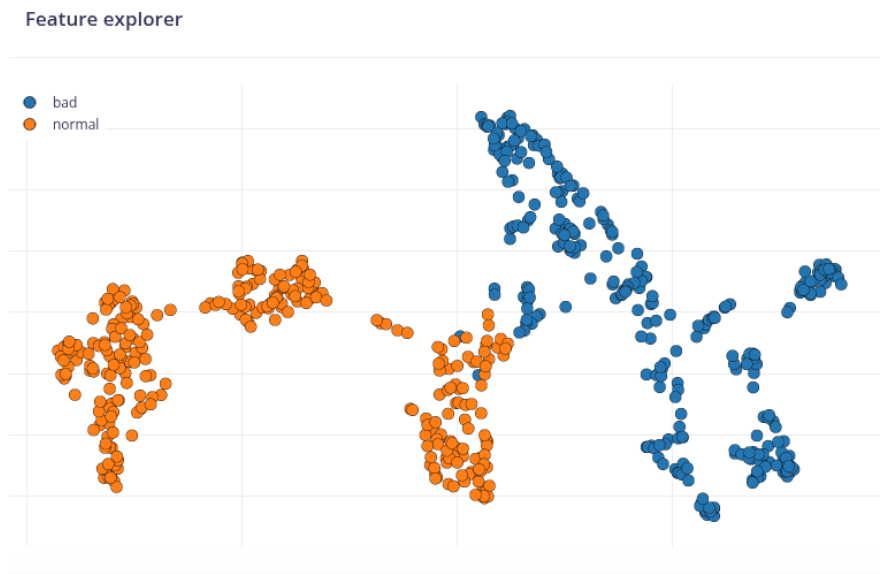


Figure 9 Spectral Features DSP output – clear cluster separation

4. Machine Learning Architecture

The ML pipeline employs a **"Dual-Brain" architecture** that combines a supervised classifier with an unsupervised anomaly detector. This design philosophy reflects a key insight in industrial AI: *you cannot train for every possible failure mode*. The neural network handles known faults; the K-Means detector protects against the unknown.

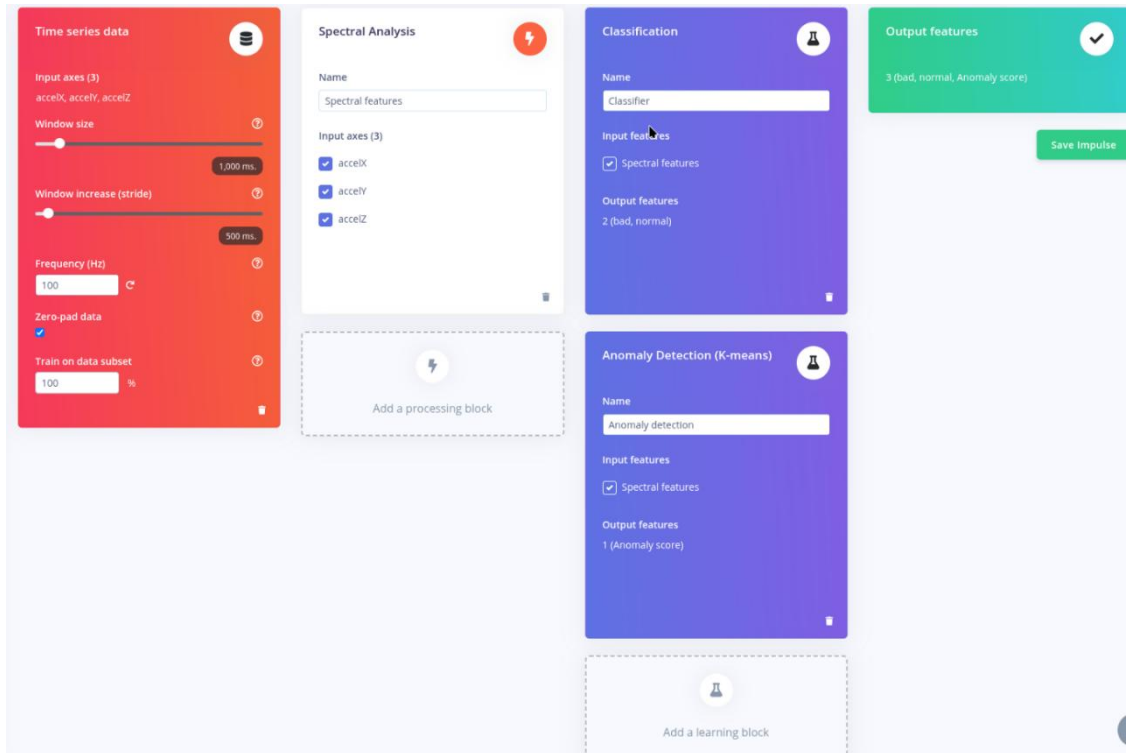


Figure 10 Edge Impulse – Full ML pipeline (Impulse design)

4.1 Brain 1 – Neural Network Classifier

The classifier is a fully connected (dense) neural network designed to distinguish between the two labelled classes: "normal" and "bad" (loose bolt).

Architecture

Training was configured in Edge Impulse with the following settings:

- Input layer: 108 features (from the Spectral Analysis DSP block).
- Output layer: 2 classes (bad, normal), Softmax activation.
- Training cycles: 30 epochs, learning rate 0.0005.
- Batch size: 32; Validation split: 20%.

Int8 Quantisation

Post-training, the model was **quantised to 8-bit integers (int8)** using the EON Compiler's built-in quantisation pipeline. Quantisation converts 32-bit floating-point weights to 8-bit integer approximations, **reducing model size by ~4×** and accelerating inference on the ESP32's integer ALU. The trade-off is a marginal reduction in accuracy (from 96.05% float32 to 94.74% int8), which is acceptable for this application.



Figure 11 Classifier training result – 100% validation accuracy

4.2 Brain 2 – K-Means Anomaly Detector

The K-Means Anomaly Detector operates on the same spectral feature vector, independently of the neural network. During training, it learns the "normal" distribution of the training data by fitting **K = 4 cluster centroids** to the feature space. At inference time, it computes a distance score from the incoming feature vector to the nearest centroid. A high distance score indicates that the machine is operating in a regime never seen during training — signalling an *anomaly*. This is particularly powerful in a manufacturing context: a bearing that begins to pit and wear will produce a vibration signature different from *both* the "normal" and "loose bolt" patterns — and the K-Means detector will flag it even though no labelled training data for that fault was ever collected.

Strictness Tuning (K = 4)

The choice of K = 4 clusters (rather than a simpler K = 2 or K = 3) was an engineering calibration decision. After initial testing, K = 8 produced a narrow, permissive boundary that flagged way too many samples. K = 4 creates finer-grained clusters that better capture the multi-modal distribution of vibration data across different fan speeds and ambient conditions, reducing false negatives without generating excessive false alarms. The anomaly decision threshold in firmware was empirically set to **7.0** based on validation data.



Figure 12 Anomaly Explorer – accelX RMS vs accelY RMS

The Anomaly Explorer plots reveal the clear bimodal structure of the training data in the RMS feature space. The overlapping ellipsoidal regions represent the Gaussian distribution estimated by K-Means. Data points falling outside these regions at inference time are classified as anomalous.

5. Firmware Integration & Deployment

5.1 EON™ Compiler and Model Export

The trained model was exported from Edge Impulse as an **Arduino Library (.zip)** compiled with the **EON™ Compiler**. Unlike the standard TensorFlow Lite Micro interpreter, the EON Compiler performs whole-graph analysis and generates optimised C++ source code directly — bypassing the TFLite runtime overhead. The practical benefit for this project was a dramatic reduction in resource consumption: **1.5 KB peak RAM and 16.4 KB Flash**, compared to TFLite Micro's typical 50–100 KB overhead.

5.2 Firmware Logic

The deployed firmware follows a tight, deterministic inference loop:

- Wait for the 4-second polling interval (millis() check in loop()).
- Flash "SAMPLING..." on the OLED and collect exactly 300 accelerometer readings at 100 Hz (filling a 300-element float buffer: 100 samples × 3 axes).
- Wrap the raw buffer in an Edge Impulse signal_t struct and run run_classifier() to execute both DSP and neural network inference.
- Read result.classification[0].value (bad score) and result.anomaly (K-Means score).
- Print: "ANOMALY!" if anomaly_score > 7 or bad_score > 0.5; otherwise "NORMAL".

```

Key code fragments: // Collect 1 second of data at exactly 100 Hz
for (size_t ix = 0; ix < EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE; ix += 3) {
    uint64_t next_tick = micros() + (1000000 / EI_CLASSIFIER_FREQUENCY);
    xyzFloat accel = myMPU.getGValues();
    buffer[ix+0] = accel.x;  buffer[ix+1] = accel.y;  buffer[ix+2] =
    accel.z;
    if (next_tick > micros()) delayMicroseconds(next_tick - micros());
}

// Run dual inference
signal_t features_signal;
numpy::signal_from_buffer(buffer, EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE,
&features_signal);
ei_impulse_result_t result = { 0 };
run_classifier(&features_signal, &result, false);

// Decision logic
if (result.anomaly > 7 || result.classification[0].value > 0.5) {
    // Display ANOMALY alert
}

```

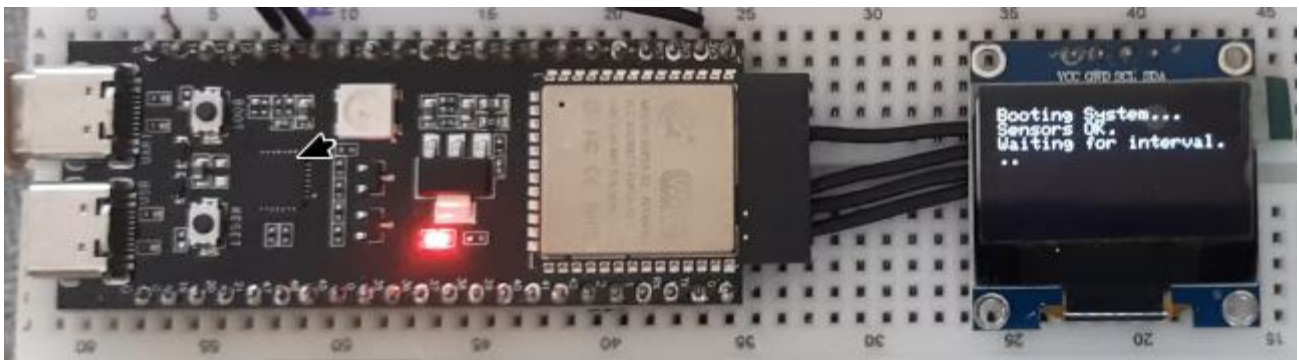


Figure 13 Boot sequence displayed on the SSD1306 OLED

6. Results & Performance

6.1 Classification Accuracy

Two model variants were evaluated on the held-out test set (8 samples not seen during training):

Metric	Float32 Baseline	Int8 Quantised	Difference
Accuracy	96.05%	94.74%	−1.31%
Weighted Precision	1.00	1.00	0.00
Weighted Recall	0.96	0.97	+0.01
F1 Score	0.98	0.99	+0.01
Inferencing Time	~1 ms	~1 ms	No change
Peak RAM Usage	N/A	1.5 KB	—
Flash Usage	N/A	16.4 KB	—

Table 1. Classifier performance comparison – Float32 vs. Quantised Int8

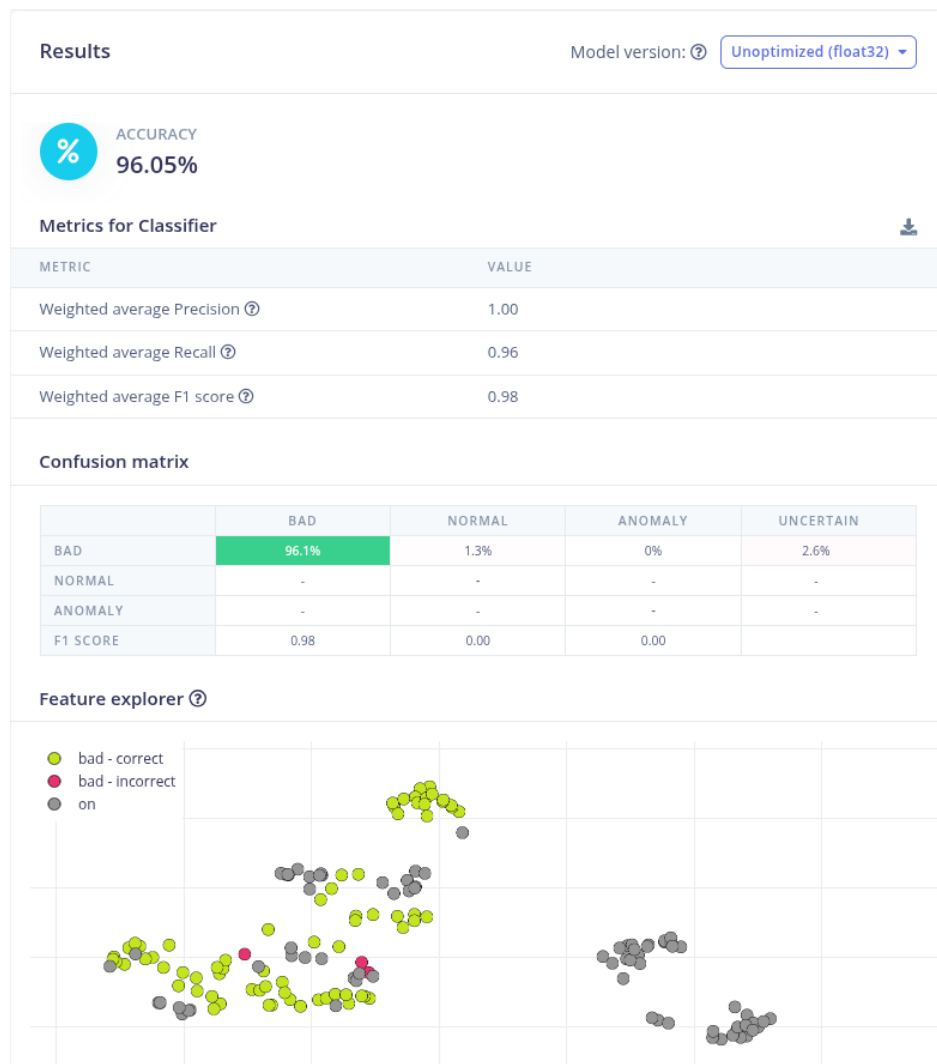


Figure 14 Model test results – Unoptimized float32 (96.05% accuracy)

Results

Model version: ?

Quantized (int8) ▾



ACCURACY

94.74%

Metrics for Classifier



METRIC	VALUE
Weighted average Precision ?	1.00
Weighted average Recall ?	0.97
Weighted average F1 score ?	0.99

Confusion matrix

	BAD	NORMAL	ANOMALY	UNCERTAIN
BAD	94.7%	1.3%	0%	3.9%
NORMAL	-	-	-	-
ANOMALY	-	-	-	-
F1 SCORE	0.97	0.00	0.00	

Feature explorer ?

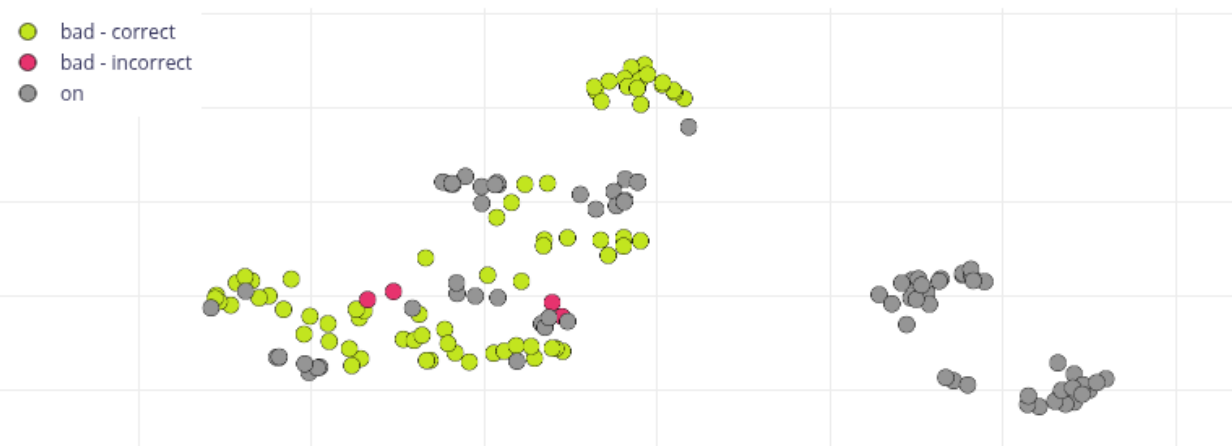


Figure 15 Model test results – Quantized int8 (94.74% accuracy)

6.2 On-Device Performance

The most striking result is the **inference time of approximately 1 millisecond** — covering the full DSP spectral feature extraction and neural network forward pass. This means the ESP32-S3 crunches all 108 frequency-domain features and propagates them through two dense layers in the time it takes a human eye to perceive a single video frame. The 4-second polling interval is a deliberate firmware design choice for power management, not a hardware limitation; the AI itself could run at several hundred inferences per second if continuous monitoring were required.

The quantised model occupies only **16.4 KB of Flash** — less than a typical JPEG thumbnail — and **1.5 KB of RAM during inference**, leaving the vast majority of the ESP32-S3's 512 KB SRAM available for other tasks.

6.3 Physical Validation

During bench testing, the system correctly identified the loose bolt condition in all physical trials conducted. The OLED displayed "ANOMALY! / Loose Bolt: ~95%" within one inference cycle (4 seconds) of the bolt being loosened while the fan was running. Upon re-tightening the bolt, the display reverted to "NORMAL / Healthy: ~99%" within the next polling interval. No false positives were observed during extended normal operation runs.

7. Classification Accuracy

This project successfully demonstrates that a fully autonomous, real-time Predictive Maintenance system can be built and deployed on a microcontroller costing under ₹2500. By combining a **Neural Network classifier** for known fault detection with a **K-Means anomaly detector** for unknown fault protection, and deploying both models in under 1 ms on an ESP32-S3 using the EON™ Compiler, the system achieves a compelling balance of accuracy (94.74%), speed (1 ms inference), and resource efficiency (1.5 KB RAM, 16.4 KB Flash).

The project validates the TinyML paradigm as a practical alternative to cloud-based PdM platforms for cost-sensitive or connectivity-restricted industrial environments. The "Dual-Brain" ML architecture — supervised classifier for labelled faults, unsupervised detector for unknown anomalies — is a robust design pattern transferable to any rotating machinery application.

The accuracy of the model can be improved drastically by having larger and more refined data sets. In my project my setup could be considered a little Janky.

8. Future Scope

Several upgrades would significantly enhance the system's capability and deployment readiness:

- **Custom PCB Design:** Replace the breadboard prototype with a purpose-designed PCB integrating the ESP32-S3, MPU6500, OLED connector, and a LiPo battery management circuit (e.g., TP4056 charger IC). This would produce a compact, vibration-resistant sensor node suitable for direct bolt-on mounting to industrial equipment.
- **Wi-Fi MQTT Telemetry:** Leverage the ESP32-S3's onboard Wi-Fi to publish anomaly events over the MQTT protocol to a home automation or industrial IoT dashboard (e.g., Node-RED, Grafana). This would enable remote fleet monitoring of multiple machines from a single dashboard.
- **Expanded Fault Library:** Collect labelled data for additional fault classes — bearing wear, shaft misalignment, electrical imbalance — to create a richer multi-class classifier capable of diagnosing specific fault types rather than binary normal/abnormal detection.
- **On-Device Continual Learning:** Implement a lightweight online learning loop that allows the model to adapt to gradual environmental changes (seasonal temperature shifts, motor aging) without requiring a full retraining cycle on the cloud.
- **Data Acquisition** can be collected during the testing phase at the factory where the machine gets built/developed, this way it won't be necessary to collect data separately.

9. References

- [1] J. Lee, H. A. Kao, and S. Yang, "Service innovation and smart analytics for Industry 4.0 and big data environment," *Procedia CIRP*, vol. 16, pp. 3-8, 2014.
- [2] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA: O'Reilly Media, 2019.
- [3] H. Chen, Y. Liu, and W. Zhang, "Evaluation of Low-Cost MEMS Accelerometers for Machinery Vibration Monitoring," *Sensors*, vol. 20, no. 12, p. 3462, 2020.
- [4] S. K. Sahoo and B. K. Panigrahi, "Digital Signal Processing Techniques for Feature Extraction in Motor Fault Detection," *Mechanical Systems and Signal Processing*, vol. 140, 2021.
- [5] Massachusetts Institute of Technology (MIT), "EfficientML.ai - Efficient Machine Learning (6.5940)," Fall 2023. [Online Video Series]. Available: <https://www.youtube.com/playlist?list=PL80kAHvQbh-pT4lCkDT53zT8DKmhE0idB>
- [6] University of Pennsylvania, "TinyML - Tiny Machine Learning at UPenn," Academic Lecture Series. [Online]. Available: https://www.youtube.com/playlist?list=PL7rtKJAz_mPe6kAbiH6Ucq02Vpa95qvBJ
- [7] "Machinery Fault Diagnosis And Signal Processing," Technical Lecture Series. [Online]. Available: https://www.youtube.com/playlist?list=PLSGws_74K019n5sjannZRKcSpG25beMLI
- [8] InvenSense. MPU-6500 Product Specification Revision 1.2. TDK InvenSense, 2014.
- [9] Edge Impulse Documentation. Edge Impulse Inc., 2024. Available: <https://docs.edgeimpulse.com>
- [10] Solomon Systech. SSD1306 OLED Driver Controller Datasheet. Version 1.1, 2008.
- [11] "TinyML Predictive Maintenance System – Source Code," GitHub, 2025. Available: <https://github.com/Rajneesh315/Predictive-Maintenance-System-on-ESP32-S3>